Lab 1: R and R Studio

NAME:

This lab will help introduce you to using R and RStudio. Luckily many of your programming skills in python and julia will transfer easily to R! We encourage you to run through everything in this lab even if it is review.

R Studio

First, make sure that R (www.r-project.org) and RStudio (https://www.rstudio.com/) are installed on your computer. Open RStudio. By default there are four panes displayed: the source pane for editing and running files, the console pane to run R commands, the environment/history/connections pane, and the files/plots/packages etc pane. We will focus on the first two although the other two are useful for checking the current status of your R session.

The **R console** executes commands as you input them. You can test out the following commands

```
x <- 3
x
```

The first command assigns the value of 3 to the variable x and the second prints out the value of x. The <- operator is the assignment operator but you can always use = instead.

```
x = 3
x
```

In your Environment pane you should now see a variable x with value 3.

If we want to write, run, and save an R script (such as for a hw assignment), we can use the **Source** pane. First, go to File -> New File -> R Script. Before we save this file, let's check the current working directory by running the following command in the console

```
getwd()
```

You can use the command **setwd()** to then change the working directory. Whenever you have questions about a command you can use a ? to get help

```
?setwd
```

This will bring up the R documentation in the Help pane. Set your working directory and then save your empty R script "Lab1.R". Now let's fill in the script with the following code

```
# This is a comment
x = 5
y = x*(x+1)
```

Note that the first line starts with a # sign indicating that it is a comment. It is always good practice to comment your code as you go. We can run a single line of the script by placing the cursor on that line and hitting the Run button in the Source pane. Try this out with all the lines above - you should see x and y be updated in your Environment pane. We can also highlight all three lines and hit Run once to have the same effect. Or hitting the Source button in the Source pane. Notice what is happening in your Console pane as you do these.

R has many built-in functions, but there are also many useful packages that we will be using along the way. To install a package you can use the install.packages command and then load the package using the library command:

```r
install.packages("dslabs")
library("dslabs")
```

Data Types

All objects in R have a type. We can check the type of an object by running the command **class**. What is the class of x?

The very basic types are **numerics**, **characters** (e.g. "hello" or "a"), and **logicals** (TRUE or FALSE). Create three variables named a, b, and c such that a is a numeric, b is a character, and c is a logical. Be sure to check the types are correct using the class command. The capitalization of TRUE and FALSE are necessary for the variable to be interpreted correctly.

We can also create **vectors** of these types. A vectors entries must all be the same type. One way to create a vector is by using the c command (standing for concatenation). For example, we can create a vector v and then add the first and second elements in two different ways:

```r
v = c(12,3,15.4)
v[1]+v[2]
sum(v[1:2])
```

Note that indices in R start from 1! We can also create a new vector of logicals that indicates whether each value in v is at least 4:

```r
u = (v >= 4)
u
```

One useful way to create a vector is to use the **seq** command. Try creating a call to seq that replicates the following output

```r
2:10
```

Often times we will work with categorical variables that can take on one of set of values. For example, suppose we are recording grades. The following code takes a vector of grade values and transforms them into a **factor**

```r
grades = c("A","C","B","C","B","A","A")
class(grades)
grades = factor(grades)
class(grades)
```

A factor is a type of variable that can only take on certain values. The command **levels** outputs the different possible values of the factor. However, F is also a possible value for a grade. We can add that by using the levels parameter of the factor command

```r
grades = factor(grades,levels=c("A","B","C","F"))
levels(grades)
```

Beyond vectors, we may need to store values in a matrix. We can create and index a matrix using the following commands

```r
entries = c(2,3,1,6,4,5)
matrix(entries,ncol=3,nrow=2)
```

Try playing around with the dimensions of the matrix. Also, try to create a matrix of all **NA** values (NA is used to represent missing or null values in R). Often it can be useful to create a vector or matrix of all zeros. We can do so easily using the following commands

```r
numeric(3)
matrix(0,nrow=2,ncol=3)
```

The first creates an all zero vector of length 3 and the second creates an all zero matrix with 2 rows and 3 columns.

The last main data type is a **list**. Unlike a vector, the entries of a list can be different types and each entry is named. We can create a list using the **list** command similar to how we used the c command earlier.

```
l = list("year" = 2010, "grades" = c("A","B","B","C"), "names" = c("Alice","Bob","Carol","Denny"))
```

The command above creates a list with three components: the year, a vector of grades, and a vector of names. To access these elements we use the symbol $ (called the accessor)

```
l$year
l$names
l$grades[1]
```

A **data frame** is a particular type of list. We will discuss data frames more later on in the lab.

Logic and Loops

To write logical statements, you can use & for "and", | for "or", and ! for "not". Test these out using the following commands

```
x = 2
y = 3
(x >= 3) & (y >= 3)
(x > 3) | (y == 3)
!(x < 3)
```

We can also apply logic to a vector. What is the output for the following command?

```
l$names[l$grades == "C"]
```

The command returns the names of students that have a "C" grade. First, l$grades == "C" returns a vector of logicals. This vector can be used to index the vector of names.

Logic can also be useful to implement if/else statements:

```
name = "Alice"
if ((name == "Alice") | (name == "alice")) {
  print("Hello, Alice!")
} else if ((name == "Bob" ) |  (name == "bob")){
  print("Hello, Bob!")
} else {
  print("Nice to meet you!")
}
```

The curly braces here are important to remember! They are also used in loops of R. Below we give an example of a for loop and a while loop:

```
x = 0
for (i in 1:10){
  x = x + i
}

x = 0
while(x < 10){
  x = x + 1
}
```

Try out your new skills by creating a script that first sets a variable x to 1. While x < 100, add 1+2+...+x to x by using a for loop. Last, if the final value is even, print "Even". Otherwise, print "Odd". To do so, you may want to use the mod %% command.

Functions

If we have a chunk of code that we want to run multiple times, it can be useful to create a function. What does the function below do?

```
multiply <- function(a,b){
  return(a*b)
}
```

The code above creates a function that takes in two numbers a and b and multiplies them. What if the user accidentally inputs a variable that is not a numeric? It is always useful to comment your code. Google's R Style Guide recommends the following comment formatting:

```
multiply <- function(a,b){
  # Args:
  #   a: a numeric value
  #   b: a numeric value
  #
  # Returns:
  #   a times b
  a*b
}
```

Try writing and commenting a function that takes in a vector of numericals and returns the sum of the entries.

Working with Data

Now we will finally load some actual data into R. We will use a dataset from the dslabs package we loaded earlier

```
library("dslabs")
dim(us_contagious_diseases)
names(us_contagious_diseases)
us_contagious_diseases$year
head(us_contagious_diseases)
```

The **dim** command tells us that there are 18,870 rows of data and 6 columns, and the **names** command tells us the names of the columns. Note that data frames are lists so we can access each column as before using the $ symbol. The last command prints the first several rows of the data. If we wanted to save this data, we can write the data to csv file.

```
write.csv(us_contagious_diseases,"diseases.csv",row.names=FALSE)
```

The row.names = FALSE states that we do not want to add an index column. Run the above command and create a data frame variable called diseases by loading diseases.csv using the **read.csv** command. We can then use the **attach** command to make it easier to access the variables without always using $.

```
attach(diseases)
year[1:4]
```

We can explore each column using the **summary** commands:

```r
summary(year)
summary(state)
summary(disease)
summary(count)
summary(weeks_reporting)
summary(population)
```

How do the outputs differ between year and state? What do you notice about the summary for population? This is the only column with some missing values. How would you summarize what information this dataset contains? We can get more info using the help command again:

```r
?us_contagious_diseases
```

When working with data, we will often want to use the **dplyr** package, which allows for easy manipulation of our data.

```r
install.packages("dplyr")
library("dplyr")
```

Suppose that we wanted to add a column that indicates whether each entry was in New England. We can do so using the **mutate** command

```r
ne = c("Maine","Vermont", "New Hampshire","Massachusetts","Rhode Island","Connecticut")
diseases = mutate(diseases, newengland = (state %in% ne))
names(diseases)
```

Note that we used %in% to test for whether an entry was in a vector. Now suppose that we actually don't need the newengland column. We can easily remove it from the dataset using the **select** command:

```r
diseases = select(diseases,-c(newengland))
names(diseases)
```

Here we used - to indicate which columns to *not* select. We could also have run the following command.

```r
diseases = select(diseases,c(disease,population,state,year,weeks_reporting,count, population))
```

We can also select certain rows by using the **filter** command.

```r
filter(diseases,state == "Rhode Island")
filter(diseases,(state == "Rhode Island") & (disease == "Measles"))
```

We can also use **piping** to combine data manipulation commands together:

```r
new.data = diseases %>% filter(state == "Rhode Island") %>% filter(disease == "Measles") %>%
  select(c(year,count))
```

The pipe %>% passes the result on the left as the first argument in the function on the right. So the dataset diseases is sent to the first filter by state, the filtered dataset is then sent to the second filter by disease, and last the doubly filtered dataset is sent to the select function. All together, the command above creates a new dataset with the years and counts for measles in Rhode Island. We could also have found this result by using the following indexing, but the piping makes it clearer what is going on.

```r
new.data = diseases[((state == "Rhode Island") & (disease == "Measles")),c("year","count")]
```

Here, we index the rows by those entries corresponding to measle counts in Rhode Island and we index the columns by year and count.

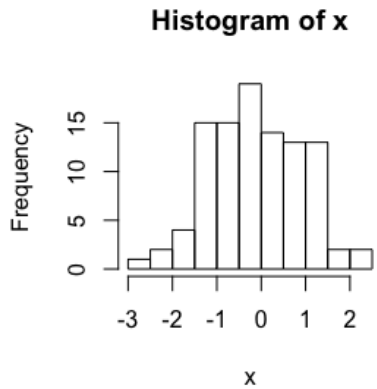

Figure 1: This is a caption

R Markdown

You may notice that this lab is also available as a .Rmd file. This is an R Markdown file. We recommend that you do your homework using this format. Download the .Rmd file and the lab1.png file. Open the .Rmd file for this lab with R Studio. The header that is on lines 1-9 tells the compiler the title of the document and any style options. You may reuse this header for any file you create (just be sure to change the title).

You can write text in R Markdown as you would in any text editor and then use the following formatting options:

- Headers: To create a section header you can use # (see line 19 where we create the R Studio section). The number of #'s determines how big the font is - the more #'s the smaller the font.
- Bold/Italics: We can *italicize* or **bold** font
- Ending a line: To create a new paragraph we can either leave a blank line
- Creating a list: We just used this option!

To create a pdf file from a .Rmd file you press the Knit to Pdf option under the Knit dropdown menu in the Source window. Try doing this with this lab.

One of the most useful parts of using R Markdown is that we can add in R code directly to the file. To do so, you can use the R option under the Insert dropdown menu in the Source window. Create an R chunk below that sets a variable x to 2 and then prints x. Knit the file and look at the output. You may notice that the other R chunks in this file have the added option eval = FALSE. This tells the compiler to display the chunk as code but not to run it. This can be useful for homework assignments when you don't want to rerun your code every time you knit your write-up.

We can also input figures into our file as below with lab1.png.

Last, R Markdown also allows us to write equations easily such as $x = y + \sqrt{2}$ or

$$z = \frac{2}{3}x^2.$$

If you are familiar with LaTeX then this format should look familiar. If not, we strongly encourage you to learn this on your own. We have provided a tutorial under the Resources folder on Canvas.

Resources

As you program for this course, you may need help finding certain functions or reviewing some R formatting. We recommend the following resources for when you get stuck:

- Labs - we will try to introduce most of the necessary functions in lab
- Introduction to Statistical Learning - the course book also has a lab at the end of each chapter
- 1010's Julia-Python-R cheatsheet - http://www.math.brown.edu/~sswatson/classes/data1010/pdf/jpr-cheatsheet.pdf
- Introduction to Data Science - https://rafalab.github.io/dsbook/
- Markdown cheatsheet - https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet
- While Googling homework questions is against the academic integrity code, it is perfectly fine to search for things like "R linear regression function." If you use code chunks to implement specific tasks for a data analysis, you must properly attribute them.